

Security Audit

Based Loans (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	6
Security Rating	7
Intended Smart Contract Functions	8
Code Quality	9
Audit Resources	10
Dependencies	10
Severity Definitions	10
Status Definitions	11
Audit Findings	12
Conclusion	45
Our Methodology	46
Disclaimers	48
About Hashlock	49

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Based Loans team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Based Loans is a fixed-term, zero-liquidation lending protocol on Base that lets users borrow USDC against supported crypto assets without liquidation risk during the loan term. The protocol is designed around deterministic loan outcomes, transparent collateral accounting, and fully on-chain settlement.

Borrowers deposit supported collateral and receive USDC with a fixed buyback price and expiry date. If the borrower buys back before expiry, the collateral is returned. If the borrower does not buy back by expiry, the collateral becomes claimable by the lenders who funded the loan. The protocol does not rely on liquidation bots, health factors, or discretionary collateral sales.

Based Loans is built to support both blue-chip and long-tail assets, giving projects and token holders access to USDC liquidity through a simpler pawn-shop-style lending model. Lender liquidity is routed through Monstro USDC where possible, allowing idle USDC to remain productive while waiting to be matched into loans.

Project Name: Based Loans

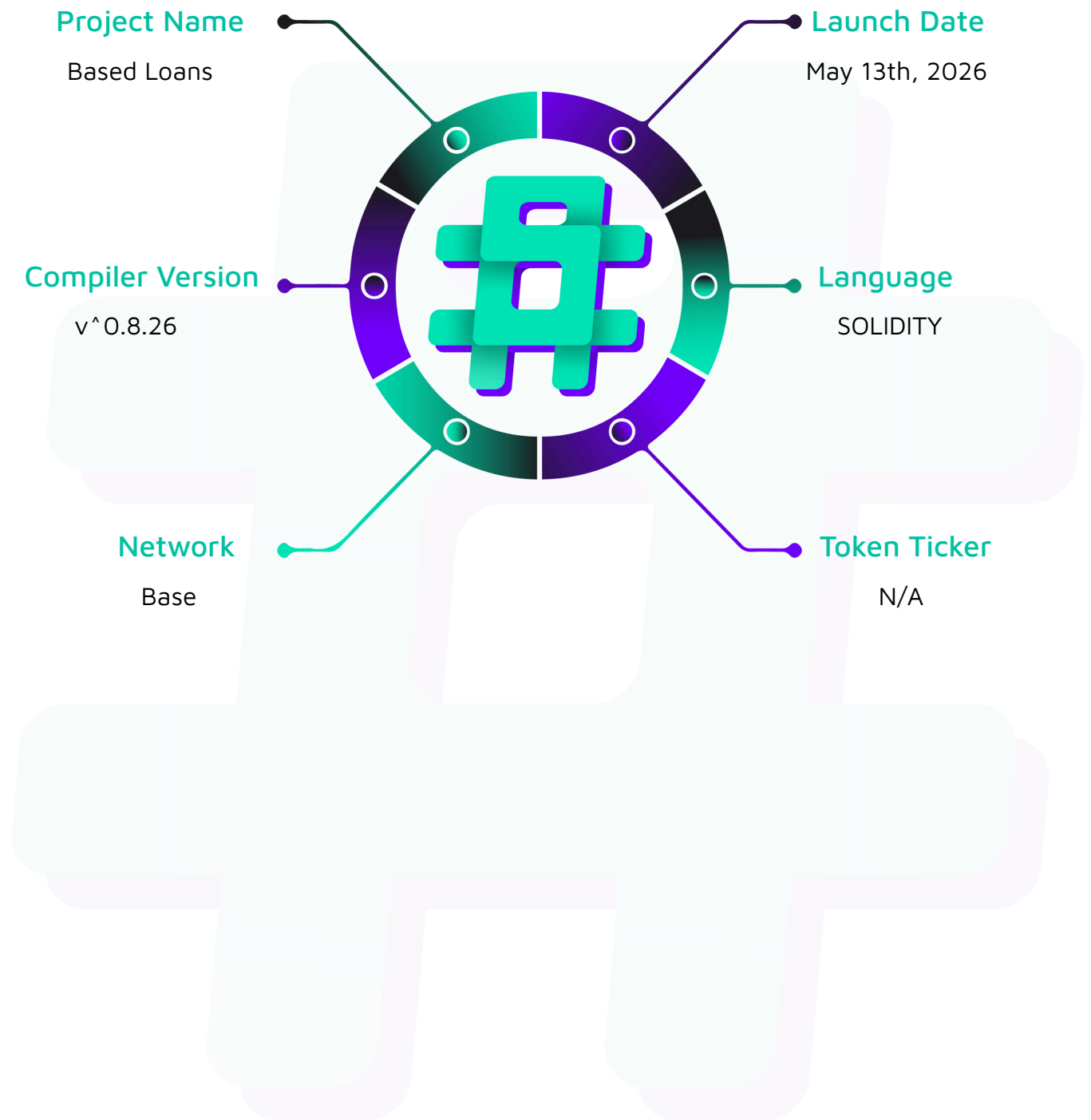
Project Type: Defi

Compiler Version: ^0.8.26

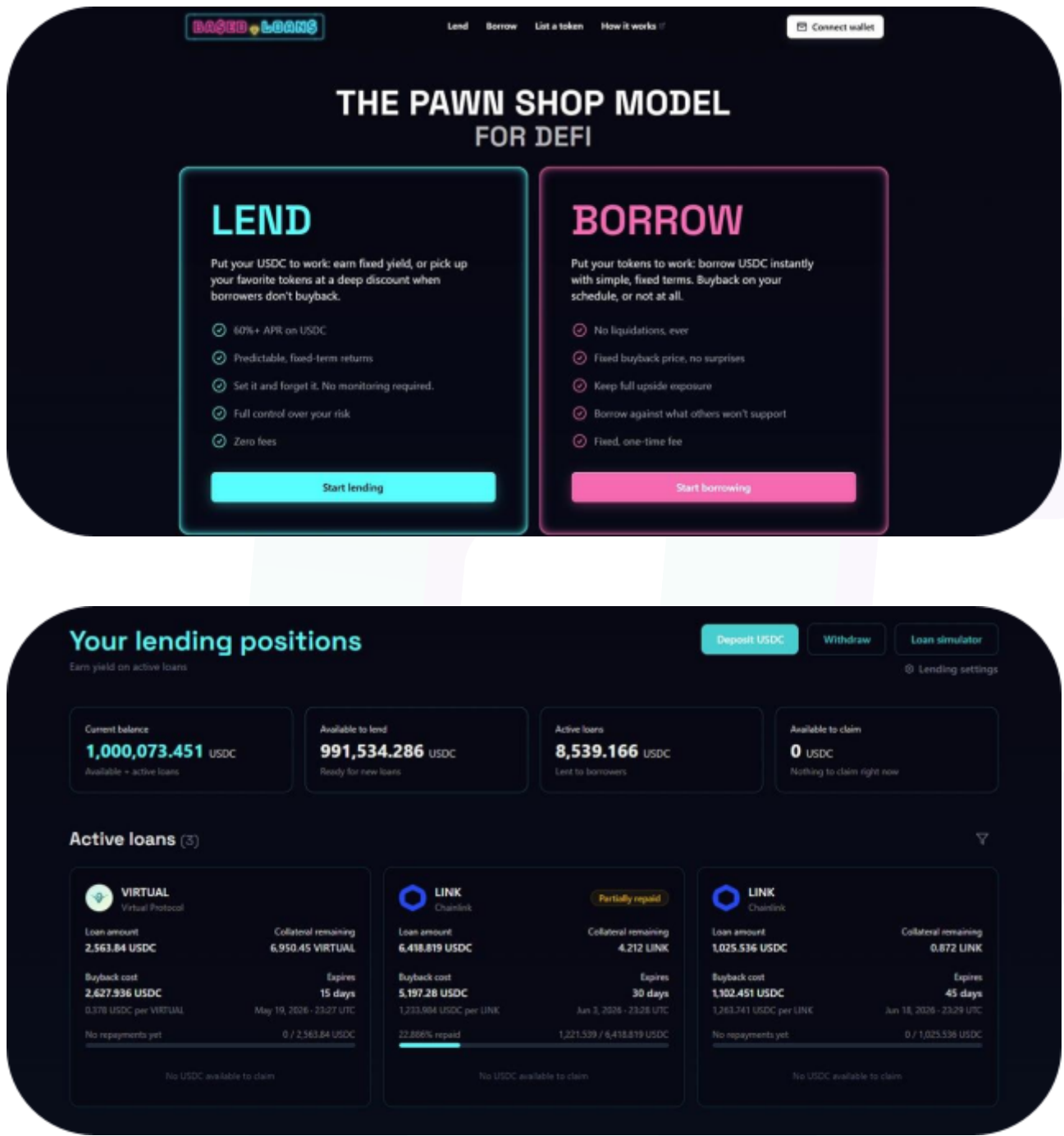
Website: <https://based.loans>

Logo:



Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the Based Loans project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Based Loans Smart Contracts
Network	Ethereum
Language	Solidity
Audit Date	April, 2026
Contract 1	Pyth.sol
Contract 2	UniswapV3.sol
Contract 3	Algebra.sol
Contract 4	LendingLedger.sol
Contract 5	Core.sol
Contract 6	AutoBurnSplitter.sol
Contract 7	AssetManager.sol
Contract 8	FeeDistributor.sol
Contract 9	OracleManager.sol
Audited GitHub Commit Hash	6f2aee66b221499cd2bc16e541647677ef8e0954
Fix Review GitHub Commit Hash	95c3036daae2333cbe3f8e51dec6ac1dbe88ea11

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved.

Hashlock found:

8 High severity vulnerabilities

11 Medium severity vulnerabilities

5 Low severity vulnerabilities

4 QA

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Code Quality

This audit scope involves the smart contracts of the Based Loans project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Based Loans project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-severity issues should be resolved before deployment. While they do not typically lead to a complete loss of funds, they may result in partial loss of funds or unintended behavior under certain conditions.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] UniswapV3Adapter#_getTwapPrice / AlgebraAdapter#_getTwapPrice

- Inverse pricing applies token decimals in the wrong direction

Description

The inverse price path uses the wrong decimal ratio. When the adapter prices `token1` -> `token0`, it should invert both the raw pool price and the token decimal adjustment. Right now it only inverts the raw price.

Vulnerability Details

For a concentrated-liquidity pool, the forward price is:

```
price = poolPrice * token0Decimals / token1Decimals
```

So the correct inverse price is:

```
price = 1 / (poolPrice * token0Decimals / token1Decimals)
      = token1Decimals / (poolPrice * token0Decimals)
```

However, the current implementation effectively does:

```
price = (1 / poolPrice) * token0Decimals / token1Decimals
      = token0Decimals / (poolPrice * token1Decimals)
```

This is a different formula.

For example, if the pool is `WETH / USDT`:

- `WETH -> USDT` is fine
- `USDT -> WETH` becomes badly mis-scaled because USDT has only 6 decimals

This can also happen in a hop path. For example, if a route uses `USDT -> WETH -> USDC`, the intermediate hop can become badly distorted when the inverse side is used.

Impact

Collateral may be heavily overvalued or undervalued. In the dangerous case, borrowers can open loans against much less real collateral than needed.

Recommendation

When taking the inverse path, invert the decimal ratio together with the raw price.

Status

Resolved

[H-02] BasedLoansLendingLedger#disburseLoan - Redeem-fee socialization is not actually applied to lender balances

Description

`disburseLoan()` reduces `totalDeposited` to account for redeem-fee loss, but it does not reduce any lender-level balances. The comments say that loss will be socialized later, but there is no later step that actually does this reconciliation.

Vulnerability Details

When the ledger unwraps vault shares to fund a loan, more shares are burned than the principal amount alone would require. The extra burn is treated as redeem-fee loss and is removed from `totalDeposited`.

The problem is that only the global variable is updated. None of the lender-level accounting is touched:

- `globalDeposit`
- `globalLocked`
- `_loanSharesOriginal`
- `_loanSharesRemaining`

This means the system now has two different views of reality:

- the global accounting says some value has already been lost
- the lender accounting still acts as if that value exists

Over time, this creates drift between aggregate state and per-user state.

Impact

Loss is not shared the way the code intends. Some lenders, especially later withdrawers, can end up bearing more loss than they should. It also makes settlement logic harder to reason about because the global and local balances no longer match.

Status

Resolved

[H-03] BasedLoansLendingLedger#requestLiquidity - FIFO queue breaks once any downstream node is dequeued

Description

If a lender is skipped because `canLend < minFillPerLender`, the queue walk moves past that lender without moving the queue head. The skipped lender remains the official head. If any downstream lender in the queue is later dequeued and their links cleared, the queue becomes corrupted because the stuck head still points through nodes that have been unlinked.

****Vulnerability Details:****

The risky branch is:

```
else if (minFillPerLender > 0 && canLend < minFillPerLender) {
    walkPtr = next;
    headAdvances = false;
}
```

At this point:

- the skipped lender becomes stuck as the official head with `nextInLine` still pointing forward
- traversal continues to later lenders via `walkPtr`
- later nodes in the queue may be dequeued and have their `nextInLine` links cleared

Once any downstream node is dequeued, the queue is broken. When the stuck head eventually tries to advance, or any future traversal attempts to follow the chain from the head, the traversal encounters nodes whose `nextInLine` has been cleared, causing the queue to break with invalid pointers.

Impact

- The queue becomes corrupted and cannot be properly traversed
- A malicious lender can trigger this by staying in the queue with `canLend < minFillPerLender` while other lenders later exit

Status

Resolved



[H-04] BasedLoansLendingLedger#disburseLoan - Redeem fees are double-counted

Description

The function estimates redeem-fee loss using a share conversion done after the vault state has already changed. Because the exchange rate is now different, the calculation overstates the fee.

Vulnerability Details

The flow is:

1. withdraw `amount` from the vault
2. compute `expectedShares = convertToShares(amount)`
3. treat `sharesBurned - expectedShares` as fee loss

The issue is that step 2 happens after step 1. Once the withdrawal has happened:

- the exchange rate is no longer the same as it was before the withdrawal

So `expectedShares` is now based on the wrong state.

Simple example:

- `totalSupply = 100`
- `totalAssets = 100`
- withdraw `50`
- actual `sharesBurned = 50.125`
- actual fee = `0.125`

After the withdrawal, the code recomputes:

- `expectedShares = 49.875`

So it calculates:

- `feeSharesCalculated = 50.125 - 49.875 = 0.25`

That is twice the real fee in this example.

Impact

The system socializes more loss than the vault actually charged.

Recommendation

Compute the fee using the same pre-withdraw state that produced `sharesBurned`, or derive it directly from the vault fee formula.

Status

Resolved

[H-05] BasedLoansLendingLedger#settleBuybackForLender - Dust sweeping can clear a lender position before the loan is fully repaid

Description

If the remaining position is small enough, `settleBuybackForLender()` clears all remaining shares even though the loan still has unpaid debt.

Vulnerability Details

The function contains this branch:

```
if (afterRelease > 0 && afterRelease <= _usdcToShares(1e6)) {  
    principalShares = remainingShares;  
}
```

That means a small leftover balance is treated as if the lender's position is finished.

The problem is that the loan itself may not be finished. Once `_loanSharesRemaining` is cleared early, later logic loses the exact share record for that lender. If the loan later ends in default, the system falls back to rough share conversion instead of using the original tracked amount.

For example, if the loan ends in default for the little amount left, `writeOffPosition` calculates the shares using the share value of the remaining amount and then subtracts it from state variables. This is bad because all state for that loan has already been cleared, which would cause a mismatch in user state and global state.

Impact

Later defaults can distort state and allow the lender to claim both collateral and loan for the dust amount left.

Recommendation

Only clear `_loanSharesRemaining` when the lender's position is actually complete.

Status

Resolved

[H-06] BasedLoansLendingLedger#settleBuybackForLender - Repayment accounting ignores mUSDC mint fees

Description

Borrower repayments are deposited back into mUSDC, which charges a mint fee. The ledger still credits lenders as if the full repayment had been minted into shares without any fee.

Vulnerability Details

The repayment flow is:

1. the borrower pays USDC - yield
2. `pullBuybackPayment()` deposits that USDC into mUSDC
3. the vault mints fewer shares because of the mint fee
4. `settleBuybackForLender()` credits lenders using:
 - yield shares from `_usdcToShares(yieldUsdc)`

These shares don't represent the actual shares minted by the vault, the same actually occurs with principal repayment however this deviation is covered in H-07 as it isn't due to fees alone.

So even though the vault minted less value than the gross repayment suggests, the ledger still credits lenders as if the full value was minted.

Impact

Lenders are credited too many shares. Over time, lender balances can grow beyond the real vault balance backing them.

Recommendation

Base lender credits on the actual shares minted by the repayment, net of mint fees.

Status

Resolved

[H-07] BasedLoansLendingLedger#settleBuybackForLender - Principal is restored using old share counts instead of current share reality

Description

Even if mint fees are ignored, repaying a fixed USDC amount after the vault share price has increased will mint fewer shares than were originally burned. The ledger still restores the old share amount.

Vulnerability Details

The function calculates:

```
principalShares = (originalShares * principal) / originalFunded;
```

This recreates the original share ratio from the time the loan was funded.

But mUSDC is a vault whose share value grows over time. That means the same USDC repayment will normally mint fewer shares later than it would have earlier.

So the ledger is effectively saying:

- "the borrower repaid this much USDC"
- therefore "the lender gets back this much old share value"

Those are not the same thing once the share price has gone up.

Impact

Lender balances can exceed the real vault share balance. That mismatch will surface later as withdrawal or settlement problems.

Recommendation

Use the actual shares minted by the repayment as the basis for principal restoration.

Status

Resolved

[H-08] BasedLoansCore#claimCollateral - Default collateral claims depend on claim order

Description

Collateral shares are calculated from `collateralRemaining`, not from the original collateral amount. This means earlier claimants use a larger base than later claimants.

Vulnerability Details

The function computes:

```
(loan.collateralRemaining * loan.amounts[lenderIndex]) / loan.totalFunded
```

Since `collateralRemaining` is reduced after every claim, later lenders receive their percentage of a smaller and smaller number.

So even if two lenders funded the same amount, they may receive different collateral amounts depending on who claims first.

Impact

Equal lenders can receive different payouts based only on claim order.

Recommendation

Calculate each lender's entitlement from the original collateral amount, and use the remaining balance only for dust handling.

Status

Resolved

Medium

[M-01] UniswapV3Adapter#getPrice / AlgebraAdapter#getPrice - Vault conversion assumes compatible decimals

Description

The vault conversion path assumes the vault share unit and the quote asset use the same decimal scaling, but this is not checked anywhere.

Vulnerability Details

The adapters do:

```
quoteVault.convertToAssets(1e18)
```

and then scale again by 1e18.

This only works cleanly if the vault and asset decimals line up with that assumption. If they do not, the returned price can be badly mis-scaled.

Impact

Prices can be significantly wrong when vault-based quote conversion is enabled.

Recommendation

Normalize using real vault and asset decimals, or validate that only compatible vault/asset pairs can be configured.

Status

Resolved

[M-02] UniswapV3Adapter#_getTwapPrice / AlgebraAdapter#_getTwapPrice

- Tick rounding can overstate inverse prices

Description

The adapters always round negative ticks downward. That is conservative for one price direction, but not for the inverse direction.

Vulnerability Details

The adapters do:

```
if (tickDelta < 0 && (tickDelta % twapWindow != 0)) avgTick--;
```

For inverse pricing (token1 -> token0), a lower tick can mean a higher reported price. So when the collateral is priced on the inverse side, this rounding rule can slightly overstate the collateral's value.

Impact

Collateral can be modestly overvalued on inverse-price paths.

Recommendation

Choose the rounding direction based on the pricing direction, not just on the sign of `tickDelta`.

Status

Resolved

[M-03] BasedLoansCore#openLoan - LTV is enforced on principal, not on total debt

Description

The LTV check is applied to the funded amount, but not to the final debt after the term premium is added.

Vulnerability Details

The flow is:

- collateral value -> max principal
- funded amount must stay below that principal cap
- then premium is added to get `buybackPrice`

So the borrower's real debt can end up above the configured LTV limit.

Impact

Loans can be riskier than the configured LTV suggests.

Recommendation

Apply the LTV limit to total debt, or reduce principal so that `buybackPrice` stays within the intended cap.

Status

Resolved

[M-04] BasedLoansLendingLedger#updateCap - Users can spam the queue without depositing

Description

A user can set a non-zero cap and enter the queue without having any usable balance.

Vulnerability Details

`updateCap` enqueues the lender when `newCap != 0`, but doesn't require:

- a deposit
- idle funds
- immediate cleanup when no funds exist

As a result, empty entries can be inserted cheaply and remain in the queue until a later liquidity walk removes them.

Impact

Attackers can fill the queue with empty lenders and increase matching overhead.

Recommendation

Require usable balance before enqueueing.

Status

Resolved

[M-05] BasedLoansLendingLedger#updateCap / #depositAndSetCaps / #requestLiquidity - Asset caps drift because utilization is stored in shares

Description

Caps are stored in USDC, but utilization is stored in vault shares. As the share price changes, the effective cap changes too.

Vulnerability Details

The remaining cap is calculated as:

- maxCap
- minus utilized converted back into USDC

When the vault share price goes up, the same number of locked shares represents more USDC. That means utilization grows over time even if the actual borrower position has not changed in USDC terms.

Impact

Lenders can be limited more than intended. A lender may appear to have less remaining cap than they expected.

Recommendation

Track utilization in the same unit as the cap, or snapshot the original USDC allocation per loan.

Status

Resolved

[M-06] BasedLoansLendingLedger#requestLiquidity - The last fill can ignore minAssetCap

Description

The code checks minAssetCap against canLend, but not against the final take amount.

Vulnerability Details

Once a lender passes the first check, the function does:

```
take = min(canLend, stillNeeded)
```

If stillNeeded is below minAssetCap, the lender can still be matched for that smaller amount.

Impact

This could lead to a borrower taking a lending position with dust amount from a lender.

Status

Resolved

[M-07] BasedLoansLendingLedger#redeployLoanProceeds - Partial redeploy mixes net and gross conversions

Description

The partial redeploy path compares the user's requested amount to a net-withdrawable USDC value, then converts that amount back to shares using a gross conversion.

Vulnerability Details

The flow is:

1. compute `totalUsdc = _sharesToUsdcNet(totalShares)`
2. cap the user's requested amount against that value
3. convert back with `_usdcToShares()`

Those are not the same accounting domain when redeem fees exist. A net withdrawable amount does not map cleanly back into shares through a gross conversion.

Impact

Users cannot reliably redeploy the exact amount they intend, and a full redeploy request can still leave residue.

Recommendation

Keep the calculation entirely in shares using `_usdcToShares` and `_sharesToUsdc`.

Status

Resolved

[M-08] BasedLoansLendingLedger#requestLiquidity - Locked shares are rounded down

Description

`requestLiquidity()` locks shares using `convertToShares`, which rounds down. The later withdrawal path uses `previewWithdraw`, which rounds up.

Vulnerability Details

The ledger records:

```
uint256 takeShares = _usdcToShares(take);
```

But the vault may require more shares than that to actually deliver take assets during withdrawal.

Impact

The recorded share lock can be smaller than the real share cost of funding the loan.

Recommendation

Use the same share requirement logic as the withdrawal path.

Status

Resolved

[M-09] BasedLoansLendingLedger#withdrawAsMusdc - The withdrawability check is too strict

Description

`withdrawAsMusdc()` transfers shares directly, but checks withdrawability using net redeemable USDC as if a redemption were happening.

Vulnerability Details

The function checks:

```
uint256 idleUsdc = _sharesToUsdcNet(idleShares);  
if (idleUsdc < amount) revert InsufficientUnlockedBalance();
```

But this path does not redeem. It only transfers shares. Redeem fees should therefore not decide whether the user can receive those shares.

Impact

Users can be blocked from withdrawing shares they should be able to receive.

Recommendation

Use a direct share-based check.

Status

Resolved

[M-10] BasedLoansLendingLedger#requestLiquidity - Idle liquidity is understated by using net redeemable USDC

Description

Queue matching reduces each lender's idle balance by redeem fees, even though the wider design treats those fees as socialized.

Vulnerability Details

The queue uses `_sharesToUsdcNet()` to measure idle balance. That means it treats redeem fees as if they belong to the currently matched lender.

But `disburseLoan()` later treats those same fees as system-wide loss.

So the protocol is using two different models at once:

- lender-local fee treatment in queueing
- socialized fee treatment in disbursement

Impact

Available liquidity can be understated, and the system becomes internally inconsistent about who pays redeem fees.

Recommendation

Use one model consistently. If fees are socialized, queue capacity should not be reduced with `_sharesToUsdcNet()`.

Status

Resolved

[M-11] BasedLoansLendingLedger#requestLiquidity - Cap utilization is measured differently in different paths

Description

Cap updates use gross utilization, but queue matching uses net utilization.

Vulnerability Details

`updateCap()` and `depositAndSetCaps()` compare caps against:

```
_sharesToUsdc(utilized)
```

while `requestLiquidity()` uses:

```
_sharesToUsdcNet(utilized)
```

So the same utilization value is interpreted in two different ways depending on the code path.

Impact

Per-asset caps can appear tighter in one place and looser in another.

Recommendation

Use one gross-or-net convention everywhere.

Status

Resolved

Low

[L-01] UniswapV3Adapter#getPrice / AlgebraAdapter#getPrice - Two-hop pricing forces both pools to share one TWAP window

Description

The two-hop path accepts only one `twapWindow`, even though each pool may need a different one.

Vulnerability Details

The first pool and the hop pool can have very different liquidity and manipulation characteristics, but the current encoding cannot express separate windows.

Impact

Oracle configuration is less flexible than it should be.

Recommendation

Allow each hop to specify its own TWAP window.

Status

Resolved

[L-02] AutoBurnSplitter#rescueToken - Pending burn USDC can be rescued

Description

`rescueToken()` can move any token out of the splitter, including USDC that is supposed to be burned.

Vulnerability Details

There is no restriction preventing the owner from rescuing USDC that has already been claimed from the FeeDistributor but not yet processed.

Impact

This introduces an owner-trust assumption around funds that users may expect to be dedicated to the burn path.

Recommendation

Disallow rescuing USDC, or clearly limit rescue to unrelated tokens.

Status

Resolved

[L-03] AutoBurnSplitter#_executeSwapBurn / #_getMinAmountOut - Quote pool and execution pool may differ

Description

The slippage check reads one pool, but the router may execute against a different fee tier.

Vulnerability Details

`_getMinAmountOut()` uses the configured pool, while `_executeSwapBurn()` passes `defaultPoolFee` to the router. If those do not point to the same actual pool, the slippage check is based on the wrong market.

Impact

Slippage protection can be weaker or stricter than intended.

Recommendation

Store and validate the fee tier per pool and ensure the execution path uses the same pool that was quoted.

Status

Resolved

[L-04] BasedLoansCore#buyback / #claimCollateral - assetExposure can accumulate dust

Description

Exposure is increased exactly, but reduced through repeated floored divisions. That leaves residual dust over time.

Vulnerability Details

Both buyback and default settlement subtract exposure proportionally, and both paths round down.

Impact

An asset can appear slightly more utilized than it really is.

Recommendation

Clear final residual exposure per loan instead of relying only on repeated proportional reductions.

Status

Resolved

[L-05] BasedLoansLendingLedger#depositAndSetCaps - `CapUpdated` can be emitted for a no-op

Description

`depositAndSetCaps()` emits `CapUpdated` even when `newCap == 0`, but in that path it does not actually clear or update the cap.

Vulnerability Details

Unlike `updateCap()`, this function has no zero-cap branch. It skips the update and still emits the event.

Impact

Off-chain systems can believe a cap changed when nothing changed on-chain.

Recommendation

Either reject zero caps here, or make the zero path behave like `updateCap()`.

Status

Resolved

QA

[Q-01] BasedLoansLendingLedger#disburseLoan - Full fee socialization may be economically harsh for small lenders

Description

Even if the fee socialization model were implemented correctly, it may still create poor outcomes for small lenders.

Vulnerability Details

Large disbursements can create large redeem fees. If that loss is spread across all depositors, smaller lenders can end up subsidizing the exit cost created by larger borrowers and larger matched lenders.

Impact

Smaller lenders may be forced to take up fees that almost wipe out their entire position.

Recommendation

Reconsider whether redeem-fee loss should be fully socialized, partially localized, or made more explicit to users.

Status

Resolved

[Q-02] Multiple in-scope contracts contain unused or dead variables

Description

Some scoped contracts define variables that are never used by live logic, or are exposed as configuration without affecting actual protocol behavior.

Vulnerability Details

Examples include:

- `BasedLoansLendingLedger._preferMusdc`
- `BasedLoansAssetManager.minAssetCap`
- `BasedLoansFeeDistributor.tier3Bps`
- `BasedLoansCore.PRICE_SCALE`

These values are stored or exposed, but they do not currently drive runtime behavior.

Impact

Dead variables make the codebase harder to reason and understand.

Recommendation

Remove unused variables, or clearly document that they are currently inert until they are wired into live logic.

Status

Resolved

[Q-03] BasedLoansCore#setContracts - Swapping the ledger can strand live loans

Description

Changing the ledger sets `loanStartId = nextLoanId`, which archives all earlier loans. Those loans can no longer be repaid or claimed through the Core, even if they are still active.

Vulnerability Details

After a ledger swap:

- `buyback()` reverts for older loans
- `claimCollateral()` skips older loans
- no migration path exists
- old lender accounting remains in the old ledger

So the protocol cuts off the user-facing settlement path for every live loan created before the swap.

Impact

Borrowers can lose the ability to recover collateral, and lenders can lose the ability to finish default settlement.

Recommendation

Do not archive live loans unless there is a full migration path, or a way to keep old loans settling against the old ledger.

Status

Resolved

[Q-04] BasedLoansCore#openLoan - Borrowers cannot choose their own shortfall tolerance

Description

Shortfall tolerance is global. Borrowers cannot set a stricter limit for their own loan.

Vulnerability Details

A borrower may want either a near-full fill or no loan at all, but the current flow only respects the protocol-wide `maxShortfallBps`.

Impact

Users can receive smaller loans than they expected.

Recommendation

Allow a per-loan shortfall limit capped by the global maximum.

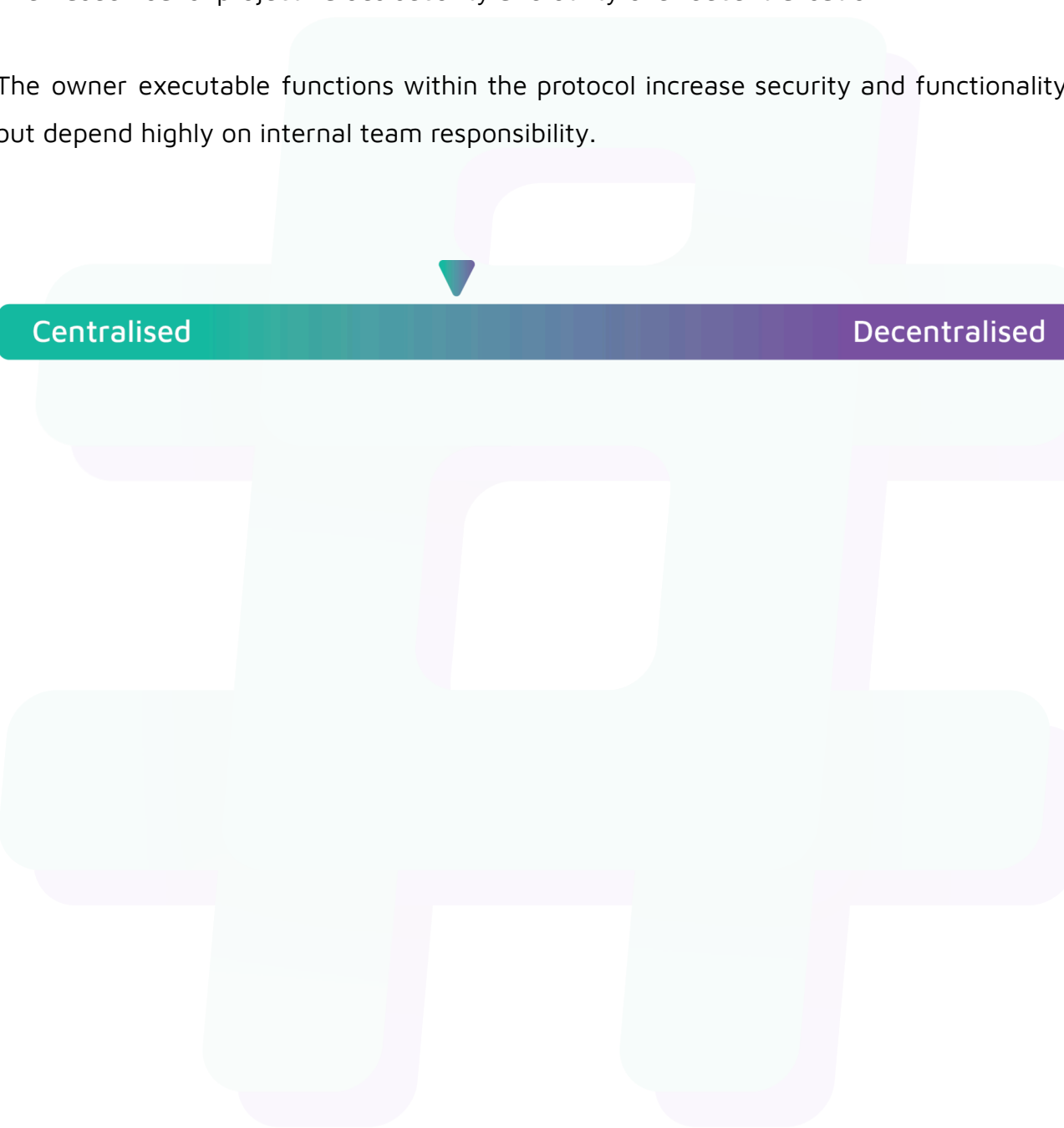
Status

Resolved

Centralisation

The Based Loans project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Based Loans project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.